

44 – AddSingleton vs AddScoped vs AddTransient

```
{
    private List<Employee> _employeeList; // field
    0 references
    public MockEmployeeRepository() // ctor
    {
        _employeeList = new List<Employee>()
        {
            new Employee() { Id = 1, Name = "Mary", Department = Dept.HR, Email = "Mary@abv.bg" },
            new Employee() { Id = 2, Name = "John", Department = Dept.IT, Email = "John@abv.bg" },
            new Employee() { Id = 3, Name = "Sam", Department = Dept.IT, Email = "Sam@abv.bg" }
        };
    }

    2 references
    public Employee Add(Employee employee)
    {
        employee.Id = _employeeList.Max(e => e.Id)+1; // max id form the list + 1 = new id
        _employeeList.Add(employee); // add new employee object to the list
        return employee; // return that object
    }
}
```

_employeeList is in-memory list = при всеки рестарт на приложението, промените в частното поле _employeeList се губят.

Когато се стартира приложението и се извика конструкторът MockEmployeeRepository(), частното поле _employeeList се инициализира с тези 3 служителя, с които започва работата му.

```
oller.cs | Startup.cs | EmployeeRepository.cs | MockEmployeeRepository.cs
EmployeeManagement.Startup | ConfigureServices(IServiceCollection)
0 references
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc(option => option.EnableEndpointRouting = false)
        .AddXmlSerializerFormatters();
    services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
    // if someone (like HomeController) requests this IEmployeeRepository service,
    // create an instance of this MockEmployeeRepository class and inject it.
    AddScoped
    AddTransient
}
```

```

public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto private field
    0 references
    public HomeController(IEmployeeRepository employeeRepository)
    {
        0 references
        //dependency injection
        _employeeRepository = employeeRepository;
    }
    0 references
    public IActionResult Index() ...
    0 references
    public IActionResult Details(int? id) ...

    [HttpGet]
    0 references
    public IActionResult Create() ...

    [HttpPost]
    0 references
    public IActionResult Create(Employee employee) //receive the employee object
    {
        if (ModelState.IsValid)
        {
            Employee newEmployee = _employeeRepository.Add(employee);
            return RedirectToAction("details", new { id = newEmployee.Id });
        }
        return View();
    }
}

```

A red arrow originates from the `IEmployeeRepository` parameter in the `HomeController` constructor and points to the `_employeeRepository` field access in the `Create` method's `Add` call.

Временно не желаем при създаването на нов служител да бъдем пренасочвани към страница Details, а да останем в същата Create, затова закоментираме пренасочването:

```

[HttpPost]
0 references
public IActionResult Create(Employee employee) //receive the employee object
{
    if (ModelState.IsValid)
    {
        Employee newEmployee = _employeeRepository.Add(employee);
        // return RedirectToAction("details", new { id = newEmployee.Id });
    }
    return View();
}

```

Red annotations highlight the changes: a red arrow points to the `employee` parameter in the `Add` call, and another red arrow points to the commented-out `RedirectToAction` line.

Освен това, искаме да ни извежда общият брой на всички служители:

```
@model Employee
[Inject] IEmployeeRepository _employeeRepository

@{
    ViewBag.Title = "Create Employee";
}

<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
    <div class="form-group row"></div>
    <div class="form-group row"></div>
    <div class="form-group row"></div>
    <div asp-validation-summary="All" class="text-danger"></div>
    <div class="form-group row"></div>
    <div class="form-group row">
        <div class="col-sm-10">
            Total Employees Count = @_employeeRepository.GetAllEmployee().Count()
        </div>
    </div>
</form>
```

[Inject] IEmployeeRepository _employeeRepository

Total Employees Count = @_employeeRepository.GetAllEmployee().Count()

Когато натиснем Create btn, и обработи заявката, нашето приложение се нуждае от IEmployeeRepository на 2 места:

- 1) в HomeController/Create [HttpPost], съответно ни трябва IEmployeeRepository service, т.е. _employeeRepository
 - 2) от същото в Create.cshtml, за да изчислим общия брой на всички служители.
- За целта към момента използваме AddSingleton in Startup.

Name	
Office Email	
Department	Please Select
Create	

Total Employees Count = 3

Какво всъщност се случва: ние сме на адрес /home/create, съответно първият [HttpGet] Create прихваща нашата заявка, а единственото, което прави, е да ни препрати в Create.cshtml. Когато разгледаме неговия код, съответно, още в началото, той има зависимост към IEmployeeRepository_employeeRepository.

В Startup имаме този код:

```
services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
```

като, вместо да се създава нова инстанция за употребата на Mock, използва отново същата IEmployeeRepository_employeeRepository в HomeController и я инжектира в Create.cshtml, който я използва, за да изчисли Count().

To serve this request an instance of the [HomeController](#) is created first. [HomeController](#) has a dependency on [IEmployeeRepository](#). This is the first time, the instance of the service is requested. So asp.net core creates an instance of the service and injects it into the [HomeController](#).

Попълваме данните веднъж и независимо колко пъти натискаме Create със същите данни, броят на служителите под него се увеличава (без да ни препраща към Details).

```
services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
```

Name	David
Office Email	david@abv.bg Singleton
Department	IT Home -> Create

Create

Total Employees Count = 4 5, 6, 7, 8, ...

3 + 1 = 4
4 + 1 = 5
5 + 1 = 6
6 + 1 = 7

AddSingleton() Създава инстанция на _employeeRepository service и се използва през целия живот от всички заявки на приложението.

* Създава се, запазва се, създава се нова, запазва се.

```
services.AddScoped<IEmployeeRepository, MockEmployeeRepository>();
```

Name	David
Office Email	david@abv.bg Scoped
Department	IT Home -> Create

Create

Total Employees Count = 4 4, 4, 4, 4, 4, ...

3 + 1 = 4
3 + 1 = 4
3 + 1 = 4
3 + 1 = 4

AddScoped() При всяко натискане на бутона, създаваме нова http заявка, която от своя страна създава нова инстанция на _employeeRepository service. Това е причината служител, създаден с една заявка, да не може да бъде видян от следващата заявка. Т.е. нова инстанция при всяка нова заявка.

* Създава се, запазва се веднъж, създава се наново всеки път.

```
services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();
```

Name	David
Office Email	david@abv.bg Transient
Department	IT Home -> Create

Create

Total Employees Count = 3 3, 3, 3, 3, 3 ...

3 + 1 = 3
3 + 1 = 3
3 + 1 = 3
3 + 1 = 3

AddTransient() всеки път се създава нова инстанция без да се запазва с всяка нова заявка.

AddSingleton AddScoped AddTransient

With a **singleton** service, there is only a single instance. An instance is created, when the service is first requested and that single instance is used by all http requests throughout the application

With a **scoped** service we get the same instance within the scope of a given http request but a new instance across different http requests

With a **transient** service a new instance is provided every time an instance is requested whether it is in the scope of the same http request or across different http requests